Diagnóstico: herramientas del navegador y tipos de error

Unidad 2. Entornos de Desarrollo, 1.º DAW

??ndice

Para qué sirve esto **1**

Parte 1. Las herramientas del navegador **3**
- 1.1. Abrirlas . . . 3
- 1.2. El mapa . . . 3
- 1.3. Elementos: el árbol que ve el navegador . . . 3
- 1.4. Cambiar cosas sin tocar el fichero . . . 4
- 1.5. Estilos: por qué no se aplica lo que has puesto . . . 4
- 1.6. El modelo de caja . . . 4
- 1.7. Consola y Red: lo que no ha llegado a cargar . . . 5
- 1.8. Vista móvil . . . 5

Parte 2. Los tres tipos de fallo **6**
- 2.1. Las tres familias . . . 6
- 2.2. En HTML la cosa cambia un poco . . . 6

Parte 3. Práctica P2.4. Seis fallos en una página **7**
- Qué te dan . . . 7
- Fase 0. Preparación . . . 7
- Fase 1. Los dos que impiden ver nada . . . 7
- Fase 2. Los otros cuatro . . . 7
- Fase 3. La tabla de diagnóstico . . . 7
- Fase 4. Una prueba en caliente . . . 8
- Qué se entrega . . . 8

Parte 4. Ejercicio de clasificación **9**
- Casos en Python . . . 9
- Casos en HTML . . . 9
- Y una pregunta final . . . 10

Resumen **11**

Para qué sirve esto

Hasta ahora has aprendido a **escribir** en VS Code. Esto va de lo otro: **averiguar por qué algo no funciona**.

Es la mitad del trabajo real de un programador y es lo que de verdad distingue a alguien que sabe usar un entorno de desarrollo. No hace falta dominar HTML ni Python para empezar: hace falta saber **dónde mirar** y **cómo se llama lo que has encontrado**.

Parte	Qué vas a aprender
1	Las herramientas de desarrollo del navegador: dónde se ve cada cosa
2	Los tres tipos de fallo y quién detecta cada uno
3	Práctica: una página con seis fallos que tienes que encontrar
4	Ejercicio: clasificar errores a partir de su corrección

Parte 1. Las herramientas del navegador

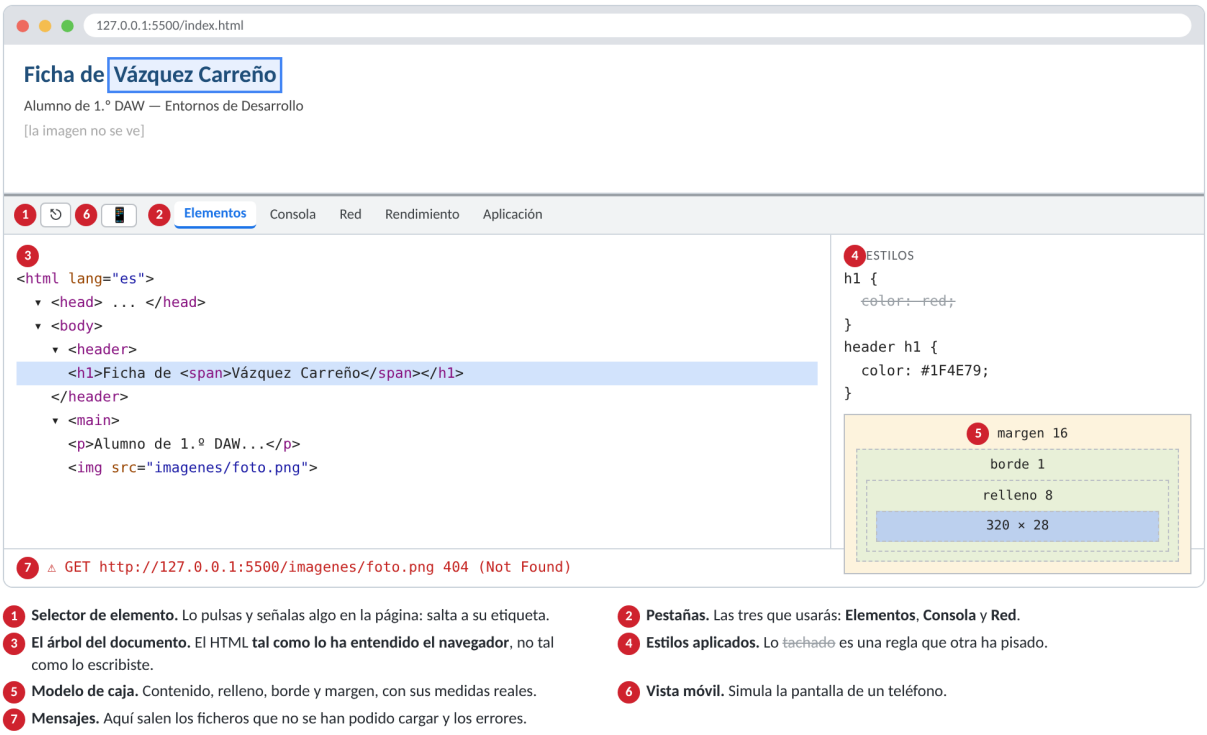
1.1. Abrirlas

Vienen incluidas en Chrome y en Firefox. No se instala nada.

Cómo	Chrome	Firefox
Tecla directa	`F12`	`F12`
Atajo completo	`Ctrl+Mayús+I`	`Ctrl+Mayús+I`
Con el ratón	Clic derecho > **Inspeccionar**	Clic derecho > **Inspeccionar**

Los dos navegadores hacen lo mismo y se parecen mucho. Cambian algunos nombres: donde Chrome pone **Elementos**, Firefox pone **Inspector**. Aquí se usan los nombres de Chrome.

1.2. El mapa



1. **Selector de elemento.** Lo pulsas y señalas algo en la página: salta a su etiqueta.
2. **Pestañas.** Las tres que usarás: **Elementos**, **Consola** y **Red**.
3. **El árbol del documento.** El HTML **tal como lo ha entendido el navegador**, no tal como lo escribiste.
4. **Estilos aplicados.** Lo ~~tachado~~ es una regla que otra ha pisado.
5. **Modelo de caja.** Contenido, relleno, borde y margen, con sus medidas reales.
6. **Vista móvil.** Simula la pantalla de un teléfono.
7. **Mensajes.** Aquí salen los ficheros que no se han podido cargar y los errores.

Figura 1: *Las siete zonas que vas a usar*

1.3. Elementos: el árbol que ve el navegador

Esta es la idea más importante de todo el documento, así que va en grande:

Para el examen / Ojo

Lo que ves en la pestaña Elementos no es tu fichero. Es el **DOM**: el árbol que el navegador ha construido después de leer tu HTML.
Si tu HTML está mal escrito, el navegador **no te avisa**: lo arregla por su cuenta como buenamente puede y sigue. Puede cerrar etiquetas que tú dejaste abiertas, o mover cosas de sitio.
Por eso, cuando algo no cuadra, lo primero es comparar las dos cosas: **tu fichero en VS Code** y **el árbol en Elementos**. Si no coinciden, ahí está el problema.

Para moverte por el árbol:

- Los triángulos ▸ abren y cierran cada rama.
- Al pasar el ratón por una línea, **se ilumina esa parte en la página**.
- Con el **selector de elemento** (el icono de la flecha, arriba a la izquierda) haces lo contrario: pulsas en la página y salta a su etiqueta.

1.4. Cambiar cosas sin tocar el fichero

Doble clic sobre cualquier texto o atributo del árbol y lo editas. La página cambia al instante.

Quiero	Cómo
Cambiar un texto	Doble clic sobre el texto y escribe
Cambiar un atributo	Doble clic sobre `src="..."` o `href="..."`
Añadir o quitar una clase	Clic derecho sobre la etiqueta > **Edit as HTML**
Ocultar un elemento	Selecciónalo y pulsa la tecla H
Borrarlo para ver qué pasa	Selecciónalo y pulsa `Supr`

Para el examen / Ojo

Nada de esto se guarda. En cuanto recargues la página (F5), todo vuelve a como estaba en tu fichero. Esto no es un fallo: es exactamente para lo que sirve. Puedes probar cinco colores, tres tamaños y dos textos en veinte segundos sin tocar el proyecto. Cuando des con el que vale, **lo escribes en VS Code**, que es donde cuenta.

1.5. Estilos: por qué no se aplica lo que has puesto

A la derecha del árbol está el panel **Estilos**, con todas las reglas CSS que afectan al elemento seleccionado, **ordenadas de la que manda a la que menos**.

Lo importante son las reglas **tachadas**: esas están escritas, el navegador las ha leído, y **otra regla las ha pisado**.

```
h1 {
  color: red;        ← tachado: esta no se aplica
}
header h1 {
  color: #1F4E79;    ← esta gana, porque es más concreta
}
```

Cuando un estilo «no funciona», el 90 % de las veces es esto: sí funciona, pero hay otra regla más concreta mandando. El panel te lo enseña sin que tengas que adivinarlo.

También puedes **desmarcar la casilla** de cualquier propiedad para ver la página sin ella, o escribir una propiedad nueva directamente ahí.

1.6. El modelo de caja

Debajo de los estilos hay un dibujo de cuatro rectángulos encajados con números. De dentro afuera:

Zona	Qué es
Contenido	El tamaño real del texto o la imagen
padding	El relleno: separación entre el contenido y el borde
border	El borde
margin	El margen: separación con los elementos de alrededor

Sirve para la pregunta de siempre: «¿por qué hay tanto hueco aquí?». Pasas el ratón por cada zona y el navegador la pinta en la página.

1.7. Consola y Red: lo que no ha llegado a cargar

La pestaña **Consola** es el buzón de quejas del navegador. Si hay un número rojo al lado del nombre, hay algo que decirte.

El mensaje más común del curso es este:

```
GET http://127.0.0.1:5500/imagenes/foto.png 404 (Not Found)
```

404 significa «ese fichero no está donde dices». Casi siempre es una ruta mal escrita: una carpeta que no existe, una mayúscula de más, un `.jpg` que en realidad es `.png`.

La pestaña **Red** enseña lo mismo pero completo: **todos** los ficheros que la página ha pedido, con su estado. Los que estén en rojo no han llegado. Recarga con la pestaña abierta, porque solo registra lo que pasa mientras está mirando.

Idea clave

Si la página se ve en blanco y negro, sin ningún estilo, no te vuelvas loco revisando el CSS: mira la Consola. Lo más probable es que el fichero `.css` no se haya cargado y el navegador esté pintando el HTML desnudo. Un 404 en la Consola te lo dice en un segundo.

1.8. Vista móvil

El icono del móvil (arriba a la izquierda de las herramientas) simula una pantalla de teléfono. Arriba puedes elegir el modelo y girar la pantalla.

No es un móvil de verdad, pero sirve para ver si algo se sale, si la letra se queda minúscula o si un menú deja de caber.

Parte 2. Los tres tipos de fallo

Para arreglar algo, primero hay que saber **de qué tipo es**, porque cada tipo se busca en un sitio distinto.

2.1. Las tres familias

Tipo	Qué pasa	Cuándo te enteras
De sintaxis	Está mal escrito para el lenguaje	**Antes de ejecutar.** El editor lo subraya, o el programa ni arranca
De ejecución	Está bien escrito, arranca, y revienta a mitad	**Mientras se ejecuta.** Sale un error y se para
De significado	Está bien escrito, arranca, termina… y el resultado es otro	**Nunca te avisa nadie.** Solo lo pilla quien comprueba el resultado

Un ejemplo de cada uno, en Python:

```
if nota >= 5            # SINTAXIS: falta el dos puntos. No llega a ejecutarse.

print("Total: " + 20)   # EJECUCIÓN: no se puede sumar texto y número. Arranca y falla.

return edad > 18        # SIGNIFICADO: funciona perfectamente,
                        # pero para 18 años contesta que no es mayor de edad.
```

Para el examen / Ojo

El tercero es el peligroso. Los dos primeros te los canta el ordenador; el tercero no lo detecta nadie salvo tú, comprobando que el resultado es el que tenía que ser.
Por eso no basta con que un programa «funcione»: hay que **comprobar que hace lo que se le pide**, con un caso del que conozcas la respuesta de antemano.

2.2. En HTML la cosa cambia un poco

HTML es más blando que Python. **No se niega a funcionar casi nunca**: si te dejas una etiqueta sin cerrar, el navegador la cierra por su cuenta y sigue. Por eso en HTML:

- Los **errores de sintaxis** no paran nada, pero deforman el resultado. Se ven con **HTMLHint** en VS Code (los subraya) y comparando el fichero con el árbol de la pestaña Elementos.
- No hay **errores de ejecución** como tales. Lo más parecido son los **404**: ficheros que la página pide y no existen.
- Los **errores de significado** son los peores de detectar: la página se ve bien, pero el enlace lleva a otro sitio, la tabla tiene las columnas cambiadas o el formulario manda el campo equivocado.

Para el examen / Ojo

Cuidado con la palabra «semántico» en HTML, que significa dos cosas distintas.

1. **Las etiquetas semánticas** son `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>`. Usar un `<div>` donde tocaba un `<nav>` es *escribir sin semántica*, pero **no es un error**: la página funciona igual.
2. Un **error de significado** es que la página haga algo distinto de lo que se pide.

Cuando en este documento se diga «de significado», se habla siempre de lo segundo.

Parte 3. Práctica P2.4. Seis fallos en una página

Qué te dan

El fichero `P2.4-ficheros.zip` contiene una página web pequeña que **se ve mal**. Tiene seis fallos metidos a propósito. Ninguno es difícil; todos se encuentran con las herramientas del navegador.

Fichero	Qué es
`index.html`	La página
`css/estilos.css`	Los estilos
`img/avatar.png`	Una imagen

Fase 0. Preparación

1. Descomprime el ZIP en una carpeta llamada `P24_TUSAPELLIDOS`.
2. Ábrela en VS Code (**Archivo > Abrir carpeta**, la carpeta entera, no un fichero suelto).
3. En `index.html`, sustituye `APELLIDOS` por tus dos apellidos de verdad. Van a salir en todas tus capturas.
4. Arranca **Live Server** y abre las herramientas con F12.

Fase 1. Los dos que impiden ver nada

Con la página abierta, mira la **Consola**. Hay dos mensajes en rojo.

1. Apunta **los dos mensajes completos**, tal cual salen.
2. Por cada uno: ¿qué fichero está pidiendo la página y dónde está ese fichero de verdad? Compáralo con el explorador de VS Code.
3. Corrígelos en `index.html` y guarda. Live Server recarga sola.

Cuando los dos estén bien, la página cambia de aspecto por completo. **Haz una captura ahora**, con las herramientas abiertas y tus apellidos visibles.

Fase 2. Los otros cuatro

Ahora que la página se ve, quedan cuatro. Cada uno se caza en un sitio distinto.

Práctica

Fallo 3. La tabla. Selecciona la tabla en la pestaña **Elementos** y despliégala entera. Compara el árbol con lo que hay escrito en `index.html`.
Pista: el navegador ha cerrado algo que tú no cerraste. ¿Qué?
Fallo 4. El identificador repetido. En VS Code, busca `id="datos"` con `Ctrl+F`. Mira cuántas veces aparece. Luego mira qué le pasa a la imagen de la segunda sección.
Pista: un `id` tiene que ser único en toda la página.
Fallo 5. El estilo que no se aplica. Selecciona el `<h1>` y mira el panel **Estilos**. Hay una regla tachada. *Apunta cuál es, cuál la está pisando y por qué gana esa.*
Fallo 6. El enlace que no lleva a ninguna parte. Pulsa los dos enlaces de la sección «Enlaces». Uno funciona y el otro recarga la página. Selecciónalo en **Elementos** y mira su atributo `href`.

Arregla los cuatro en el fichero y guarda.

Fase 3. La tabla de diagnóstico

Rellena esta tabla en un documento. **Una fila por fallo.**

Nº	Qué se veía mal	Dónde lo detecté	Qué lo causaba	Tipo
1		Consola / Elementos / Estilos / Red		Sintaxis / Ejecución / Significado
…				

En la columna **«Dónde lo detecté»** pon la pestaña concreta. Es la columna que más vale: demuestra que has usado la herramienta y no que lo has adivinado leyendo el código.

Fase 4. Una prueba en caliente

Antes de entregar, con la página ya arreglada:

1. Selecciona tu `<h1>` en **Elementos**.
2. En el panel **Estilos**, cambia el color a `#C55A11` **sin tocar el fichero**.
3. Captura la pantalla con el color cambiado.
4. Recarga con F5 y captura otra vez.

En el documento, explica en dos líneas **por qué la segunda captura ha vuelto al color anterior**.

Qué se entrega

Un ZIP llamado `P24_TUSAPELLIDOS.zip` con:

	Contenido
[ ]	La carpeta del proyecto **ya corregida** y funcionando
[ ]	Un documento con la tabla de la Fase 3, las seis filas rellenas
[ ]	Las capturas de la Fase 1 y de la Fase 4, con tus apellidos visibles
[ ]	La explicación de dos líneas de la Fase 4

Parte 4. Ejercicio de clasificación

Aquí **no tienes que encontrar el fallo**: te doy el código y, debajo, el cambio exacto que lo arregla. No te digo qué hace ese cambio ni por qué.

Tu trabajo es el de un técnico que recibe un parte de reparación: mirar el antes y el después y **decir de qué tipo era el fallo**, cómo se manifestaba y quién lo habría detectado.

Por cada caso, rellena las cuatro columnas:

Caso	Tipo	¿Qué se veía?	¿Quién lo detecta?
	Sintaxis / Ejecución / Significado	No arranca / Se para a mitad / Da un resultado equivocado / Se ve deformado	El editor / El navegador / El propio programa / Solo una persona comprobando

Casos en Python

Caso 1

```
nota = 7
if nota >= 5
    print("Aprobado")
```

> Donde pone `if nota >= 5`, pon `if nota >= 5:`

Caso 2

```
notas = [5, 7, 9]
suma = 0
for n in notas:
    suma = n
print("Media:", suma / len(notas))
```

> Donde pone `suma = n`, pon `suma = suma + n`

Caso 3

```
precio = 20
descuento = 10
print("Precio final: " + precio * (1 - descuento / 100))
```

> Donde pone `print("Precio final: " + precio * (1 - descuento / 100))`, pon `print("Precio final:", precio * (1 - descuento / 100))`

Caso 4

```
def es_mayor(edad):
    return edad > 18

print(es_mayor(18))
```

> Donde pone `return edad > 18`, pon `return edad >= 18`

Casos en HTML

Caso 5

```
<ul>
  <li>Primero
  <li>Segundo
</ul>
```

> Donde pone `<li>Primero`, pon `<li>Primero</li>` Donde pone `<li>Segundo`, pon `<li>Segundo</li>`

Caso 6

```
<img src=foto.png alt="Mi foto">
```

> Donde pone `src=foto.png`, pon `src="foto.png"`

Caso 7

```
<a href="https://www.google.com">Documentación de HTML</a>
```

> Donde pone `https://www.google.com`, pon `https://developer.mozilla.org/es/`

Caso 8

```
<table>
  <tr><th>Nombre</th><th>Nota</th></tr>
  <tr><td>8</td><td>Ana</td></tr>
</table>
```

> Donde pone `<td>8</td><td>Ana</td>`, pon `<td>Ana</td><td>8</td>`

Y una pregunta final

De los ocho casos, **¿cuáles habrías encontrado tú sin que nadie te dijera que hay un fallo?** Nómbralos y explica en una línea por qué los otros se te habrían pasado.

Resumen

Si ves esto	Mira aquí
La página sin estilos, en blanco y negro	**Consola**: busca un 404 del fichero `.css`
Una imagen rota	**Consola** o **Red**: la ruta del `src`
Un estilo que «no funciona»	**Estilos**: estará tachado, con otra regla encima
Un hueco que no entiendes	**Modelo de caja**
Algo que se coloca donde no toca	**Elementos**: compara el árbol con tu fichero
Un enlace que no va a ningún sitio	**Elementos**: mira su `href`
La página se ve bien pero dice cosas raras	Nadie te va a avisar. Compruébalo tú

Y las tres frases que hay que recordar:

1. El árbol de **Elementos** no es tu fichero: es lo que el navegador ha entendido.
2. Lo que cambias en las herramientas **no se guarda**. Sirve para probar, no para arreglar.
3. Un programa que arranca y termina **no es un programa correcto**. Solo es uno que no ha protestado.